

Generative AI Discussions on Bluesky

COMP3020 Social Web Analytics - Group 19

Group Details

Group 19

Group Member	Student ID	Contribution
Arend Chandra	[22150760]	45%
Yat Cheng	[22150815]	30%
Tim Carino	[22204733]	25%

Total contribution: 100%

Introduction

This report investigates, Public Discussion of Generative AI on Bluesky: Content, Engagement, Clustering and Interaction Network Analysis

Research Questions

Hypothesis Testing: Is receiving at least one like independent of the generative-AI topic being discussed (ChatGPT, Claude AI or Google Gemini)?

Text/content Analysis & Visualisation: What words and themes are most prominent in Bluesky discussions about generative AI

Clustering: Can generative-AI posts be grouped into meaningful clusters based on similarities in their textual content?

Network Analysis: Which users are the most structurally important within a Bluesky follow network surrounding generative-AI discussions?

Repository

GitHub repository: <https://github.com/herman402/social-web-analytics>

2.1 Data Collection

```

# File:
# 01_data_collection.R
#
# Purpose:
# Retrieve and inspect Bluesky posts for the project.
#
# Lecture / Lab Source:
# Module 4 - Text Mining 1, Part 1

# Bluesky authentication is completed locally before running this script.
# Do not store the app password in the shared GitHub repository, R code is
# sending request(s) to the Bluesky API through atrrr is working/functioning.

# Install once only if not already installed
# install.packages("atrrr")

library(atrrr)

# ChatGPT
search_skeets_ChatGPT = search_post(
  "ChatGPT",
  sort = "latest",
  since = "2025-10-04",
  limit = 200
)

# Claude AI
search_skeets_Claude = search_post(
  "Claude AI",
  sort = "latest",
  since = "2025-10-04",
  limit = 200
)

# Google Gemini
search_skeets_Gemini = search_post(
  "Google Gemini",
  sort = "latest",
  since = "2025-10-04",
  limit = 200
)

# Check the number of rows and columns
dim(search_skeets_ChatGPT)
dim(search_skeets_Claude)
dim(search_skeets_Gemini)

# Check the available variables
names(search_skeets_ChatGPT)

# Preview the first few records

```

```

head(search_skeets_ChatGPT)

# View the dataset
# View(search_skeets_ChatGPT)

# Save raw Bluesky datasets
save(
  search_skeets_ChatGPT,
  search_skeets_Claude,
  search_skeets_Gemini,
  file = "data/raw/bluesky_raw_data.RData"
)

```

The collected raw Bluesky data is loaded from the saved project file for use in the report.

```

load("../data/raw/bluesky_raw_data.RData")

# Check the number of rows and columns
dim(search_skeets_ChatGPT)

```

```
## [1] 297 23
```

```
dim(search_skeets_Claude)
```

```
## [1] 200 23
```

```
dim(search_skeets_Gemini)
```

```
## [1] 200 23
```

```

# Check the available variables
names(search_skeets_ChatGPT)

```

```

## [1] "uri"          "cid"          "author_handle" "author_name"
## [5] "text"         "author_data"  "post_data"     "embed_data"
## [9] "reply_count"  "repost_count" "like_count"    "quote_count"
## [13] "bookmark_count" "indexed_at"   "created_at"    "in_reply_to"
## [17] "in_reply_root" "quotes"       "tags"          "mentions"
## [21] "links"        "langs"       "labels"

```

Preview the first few records

```
head(search_skeets_ChatGPT) #too long
```

Data Cleaning

```
# Data Cleaning =====
# Load libraries:
library("tm")
```

```
## Warning: package 'tm' was built under R version 4.5.3
```

```
## Loading required package: NLP
```

```
## Warning: package 'NLP' was built under R version 4.5.3
```

```
library("SnowballC")
```

```
## Warning: package 'SnowballC' was built under R version 4.5.2
```

```
library("jsonlite")
```

```
## Warning: package 'jsonlite' was built under R version 4.5.3
```

```
# Load data:
load("../data/raw/bluesky_raw_data.RData")
```

```
# Text gathering:
chatgpt_text = search_skeets_ChatGPT$text
claude_text = search_skeets_Claude$text
gemini_text = search_skeets_Gemini$text
```

```
# Combine all texts into one:
all_texts = c(chatgpt_text, claude_text, gemini_text)
```

```
# Make corpus:
corpus = Corpus(VectorSource(all_texts))
```

```
# ASCII conversion:
corpus = tm_map(corpus, function(x) iconv(x, to = "ASCII"))
```

```
## Warning in tm_map.SimpleCorpus(corpus, function(x) iconv(x, to = "ASCII")):
## transformation drops documents
```

```
# URL removal:
remove_url = content_transformer(function(x) {
  gsub("https?://|www\\.\\.\\.") [^[:space:]]<>]+", "", x, perl = TRUE)
})
```

```
# Data transformation:
names = c("ChatGPT", "Claude", "Gemini")
bad_words = fromJSON("../data/words.json")
```

```
corpus = tm_map(corpus, remove_url)
```

```
## Warning in tm_map.SimpleCorpus(corpus, remove_url): transformation drops
## documents
```

```
corpus = tm_map(corpus, removeNumbers)
```

```
## Warning in tm_map.SimpleCorpus(corpus, removeNumbers): transformation drops
## documents
```

```
corpus = tm_map(corpus, removePunctuation)
```

```
## Warning in tm_map.SimpleCorpus(corpus, removePunctuation): transformation drops
## documents
```

```
corpus = tm_map(corpus, stripWhitespace)
```

```
## Warning in tm_map.SimpleCorpus(corpus, stripWhitespace): transformation drops
## documents
```

```
corpus = tm_map(corpus, tolower)
```

```
## Warning in tm_map.SimpleCorpus(corpus, tolower): transformation drops documents
```

```
corpus = tm_map(corpus, removeWords, stopwords("english"))
```

```
## Warning in tm_map.SimpleCorpus(corpus, removeWords, stopwords("english")):
## transformation drops documents
```

```
corpus = tm_map(corpus, removeWords, tolower(names))
```

```
## Warning in tm_map.SimpleCorpus(corpus, removeWords, tolower(names)):
## transformation drops documents
```

```
corpus = tm_map(corpus, removeWords, bad_words)
```

```
## Warning in tm_map.SimpleCorpus(corpus, removeWords, bad_words): transformation
## drops documents
```

```
corpus = tm_map(corpus, stemDocument)
```

```
## Warning in tm_map.SimpleCorpus(corpus, stemDocument): transformation drops
## documents
```

Inspect corpus

```
inspect(corpus) #too long
```

```
# Save processed texts:
save(corpus, file = "data/processed/bluesky_processed_data.RData")
```

2.2 Hypothesis Testing

Research Question

Is receiving at least one like independent of the generative-AI topic being discussed (ChatGPT, Claude AI or Google Gemini)?

The aim of this analysis is to test whether the AI topic of a Bluesky post is associated with whether that post received at least one like.

Hypotheses

- **H0:** AI topic and receiving at least one like are independent.
- **HA:** AI topic and receiving at least one like are associated.

Data Preparation

The raw Bluesky datasets were loaded and each post was assigned to one of the three AI topics.

```
load("../data/raw/bluesky_raw_data.RData")

topic = c(
  rep("ChatGPT", nrow(search_skeets_ChatGPT)),
  rep("Claude", nrow(search_skeets_Claude)),
  rep("Gemini", nrow(search_skeets_Gemini))
)
```

A second variable was created to record whether each post received at least one like.

```
liked = c(
  search_skeets_ChatGPT$like_count > 0,
  search_skeets_Claude$like_count > 0,
  search_skeets_Gemini$like_count > 0
)

tab = table(topic, liked)

tab
```

```
##           liked
## topic    FALSE TRUE
## ChatGPT    184  113
## Claude     144   56
## Gemini      137   63
```

The contingency table compares AI topic with whether a post received no likes (FALSE) or at least one like (TRUE).

Chi-Squared Test

A chi-squared test for independence was used because both variables are categorical:

- AI topic: ChatGPT, Claude or Gemini
- Like status: at least one like or no likes

```
chi_test = chisq.test(  
  tab,  
  simulate.p.value = TRUE  
)  
  
chi_test$observed
```

```
##      liked  
## topic  FALSE TRUE  
## ChatGPT   184  113  
## Claude    144   56  
## Gemini    137   63
```

The observed counts were:

- ChatGPT: 184 posts received no likes and 113 received at least one like.
- Claude: 144 posts received no likes and 56 received at least one like.
- Gemini: 137 posts received no likes and 63 received at least one like.

Percentage of Posts Receiving at Least One Like

```
like_percent = tab[, "TRUE"] / rowSums(tab) * 100  
  
like_percent
```

```
## ChatGPT  Claude  Gemini  
## 38.04714 28.00000 31.50000
```

The percentage of posts receiving at least one like was approximately:

- ChatGPT: 38.0%
- Claude: 28.0%
- Gemini: 31.5%

ChatGPT had the highest proportion of posts receiving at least one like in the collected sample, followed by Gemini and Claude.

Expected Frequencies

The expected frequencies represent the counts that would be expected if AI topic and like status were independent.

```
chi_test$expected
```

```
##           liked
## topic      FALSE    TRUE
## ChatGPT 198.142 98.85796
## Claude  133.429 66.57102
## Gemini   133.429 66.57102
```

The chi-squared test compares these expected counts with the observed counts in the dataset.

Test Result

```
chi_test
```

```
##
## Pearson's Chi-squared test with simulated p-value (based on 2000
## replicates)
##
## data:  tab
## X-squared = 5.8357, df = NA, p-value = 0.05497
```

The chi-squared statistic was approximately 5.84 and the simulated p-value was approximately 0.054.

```
chi_test$p.value
```

```
## [1] 0.05497251
```

Since the p-value is greater than 0.05, the null hypothesis is not rejected.

There is therefore insufficient statistical evidence at the 5% significance level to conclude that receiving at least one like is associated with whether a post discusses ChatGPT, Claude AI or Google Gemini.

Although ChatGPT posts had the highest observed proportion of posts receiving at least one like, the differences between the three topics were not statistically significant.

Limitation

The posts were collected over a short time window. Engagement may change over time as newer posts receive additional likes, so the results represent only the engagement observed during the collection period.

2.3 Text and Content Analysis

2.4 Clustering

Clustering

Brief

This analysis aims to find out if Generative AI posts from Bluesky can be grouped into meaningful clusters based on similarities in their textual content. This involves the usage of clustering methods such as K-Means

with the Elbow Method to show that clusters can be generated from the social media posts, and hierarchical clustering to further prove how those posts can be grouped into meaningful clusters at all.

K-Means Clustering

K-Means Clustering is a clustering method that allows similar data points to be grouped into separate clusters without needing much supervised data and works by identifying natural groups in any unlabelled datasets and groups points based on distance to cluster centres.

This method was applied to Bluesky posts that had discourse about Generative AI through the usage of Term Document Matrices, TF-IDF weighting, the Elbow Method, and distance calculation using Cosine Distance in order to extract as clear of clusters as possible to prove that K-Means clustering can work in this dataset of posts.

4.1) Shaping the Data To begin K-Means clustering, the processed corpus from the Bluesky posts were loaded into the environment in order to start shaping the data to be suitable for clustering. What was initially done was to create a Term Document Matrix off the corpus, then using TF-IDF weighting from the Document Matrix to better weigh word frequencies, and then shaping it to a matrix which is then stripped of empty frequencies.

This shaped data will be utilised for future steps as the shaped data will be the required type of data for input in k-means clustering.

```
# Load libraries to prepare:
```

```
library("tm")
```

```
library("dendextend")
```

```
## Warning: package 'dendextend' was built under R version 4.5.3
```

```
##
```

```
## -----
```

```
## Welcome to dendextend version 1.19.1
```

```
## Type citation('dendextend') for how to cite the package.
```

```
##
```

```
## Type browseVignettes(package = 'dendextend') for the package vignette.
```

```
## The github page is: https://github.com/talgalili/dendextend/
```

```
##
```

```
## Suggestions and bug-reports can be submitted at: https://github.com/talgalili/dendextend/issues
```

```
## You may ask questions at stackoverflow, use the r and dendextend tags:
```

```
## https://stackoverflow.com/questions/tagged/dendextend
```

```
##
```

```
## To suppress this message use: suppressPackageStartupMessages(library(dendextend))
```

```
## -----
```

```
##
```

```
## Attaching package: 'dendextend'
```

```
## The following object is masked from 'package:stats':
```

```
##
```

```
## cutree
```

```

# Load Bluesky processed data:
load("../data/processed/bluesky_processed_data.RData")

# K-Means Clustering ====
# Build matrix to be used:
text_tdm = TermDocumentMatrix(corpus)
text_wtdm = weightTfIdf(text_tdm)

## Warning in weightTfIdf(text_tdm): empty document(s): 2 3 4 6 7 8 9 11 12 13 14
## 15 16 17 19 20 21 23 25 26 27 29 30 31 32 33 36 37 38 39 40 41 42 43 44 45 46
## 48 51 52 53 54 56 57 58 59 61 62 63 65 66 67 68 69 70 76 77 78 80 82 84 85 87
## 88 89 90 91 93 94 95 98 99 100 101 102 103 104 106 107 108 112 113 114 116 121
## 122 123 124 127 128 129 130 133 139 141 143 144 147 148 150 151 152 153 154 155
## 156 157 158 159 167 170 174 175 178 180 181 183 184 185 186 187 188 189 190 191
## 192 193 196 197 198 201 204 205 207 208 210 211 212 213 215 220 224 225 226 227
## 228 229 230 231 232 234 235 236 237 238 239 240 242 243 246 247 248 252 253 254
## 255 256 258 259 260 261 264 265 269 270 271 273 274 275 276 277 278 281 282 284
## 285 286 287 288 290 291 292 293 297 300 303 305 306 307 309 311 314 319 320 324
## 325 327 328 330 333 334 335 337 341 344 347 348 349 350 351 354 355 356 357 364
## 366 367 368 372 377 378 380 382 384 386 387 388 390 392 393 396 398 400 402 403
## 404 406 407 408 410 412 414 415 416 420 421 423 428 430 431 434 436 438 439 440
## 441 442 443 446 447 449 451 452 456 457 461 462 464 466 467 471 472 474 475 476
## 477 480 481 483 484 485 486 487 488 491 492 495 499 500 501 503 509 511 515 520
## 521 522 523 526 529 530 531 532 533 535 537 540 541 542 544 545 546 547 549 550
## 551 558 561 562 564 566 568 572 573 574 579 580 581 582 586 589 590 591 592 597
## 598 600 602 603 604 605 606 607 608 609 610 611 612 613 616 617 619 620 621 623
## 625 626 627 628 631 632 633 636 637 638 639 641 642 643 644 645 646 647 648 649
## 650 651 652 653 654 655 656 657 659 661 664 665 666 668 670 673 674 676 678 679
## 680 681 682 683 685 687 690 691 692 694 698 699 701 704 706 709 712 713 716 719
## 720 721 722 724 725 726 727 728 729 730 731 732 733 735 736 737 738 740 741 743
## 744 745 746 748 749 750 751 753 755 756 758 761 762 763 766 769 773 774 776 778
## 781 782 784 786 791 794 795 796

text_matrix = t(as.matrix(text_wtdm))

# Keep non-empty
empties = which(rowSums(abs(text_matrix)) == 0)
text_matrix = text_matrix[-empties, ]

```

4.2) Elbow Method The elbow method is a visual method that is used in K-Means clustering in order to find the optimal amount of clusters (K) through how performance changes with each increasing k-value. For this dataset, the in-built K-Means function was used to check performance of K with the metric Total Within-cluster Sum of Squares (SSW) because SSW directly measures how compact the clusters are which is important to understanding if there are meaningful clusters that can be created. For this method, the matrix that was created will be used to check for clusters.

```

set.seed(69) # Set seed to get the same results every time

# Do elbow method:
n = 15
SSW = rep(0, n)

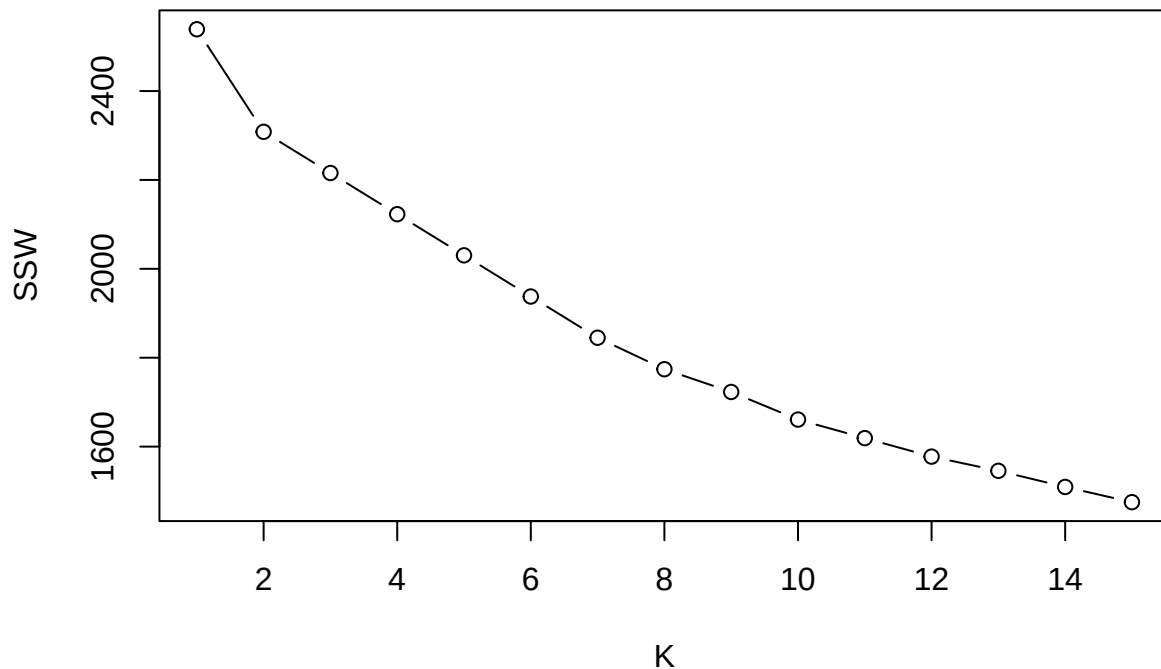
```

```

for (a in 1:n) {
  K = kmeans(text_matrix, a, nstart = 10) # K-Means
  SSW[a] = K$tot.withinss # total within-cluster sum of squares
}
# Plot elbow:
plot(1:n, SSW, type = "b", xlab = "K", main = "K-Means Elbow Method")

```

K-Means Elbow Method



As illustrated from the elbow method graph, the elbow begins to slow down at around $K = 5$, meaning that from the graph, the optimal amount of clusters for this dataset would be five clusters for the K-Means method of clustering.

4.3) K-Means with Cosine Distance Now that the amount of clusters to use for K-Means, the clustering of the Bluesky posts dataset can now be illustrated, but before that can happen - the distance has to be applied to clustering properly in order to show the clearest possible clusters possible. This can be done by first normalising document vector to unit length to prevent longer posts from appearing, then calculating cosine distance of the matrix in order to project to Euclidean geometry, which is done by classical multidimensional scaling (MDS) of the matrix of values above zero.

```

# Normalise to unit length:
normalised_matrix = diag(1 / sqrt(rowSums(text_matrix ^ 2))) %*% text_matrix

# Cosine distance:
D = dist(normalised_matrix, method = "euclidean") ^ 2 / 2

# Available unit vectors:

```

```
N = dim(text_matrix)[1]
N
```

```
## [1] 323
```

```
# MDS:
mds.matrix = cmdscale(D, k = 301) # 301/322 eigenvalues > 0
```

```
## Warning in cmdscale(D, k = 301): only 282 of the first 301 eigenvalues are > 0
```

```
dim(as.matrix(D)) # N*N document-distance matrix
```

```
## [1] 323 323
```

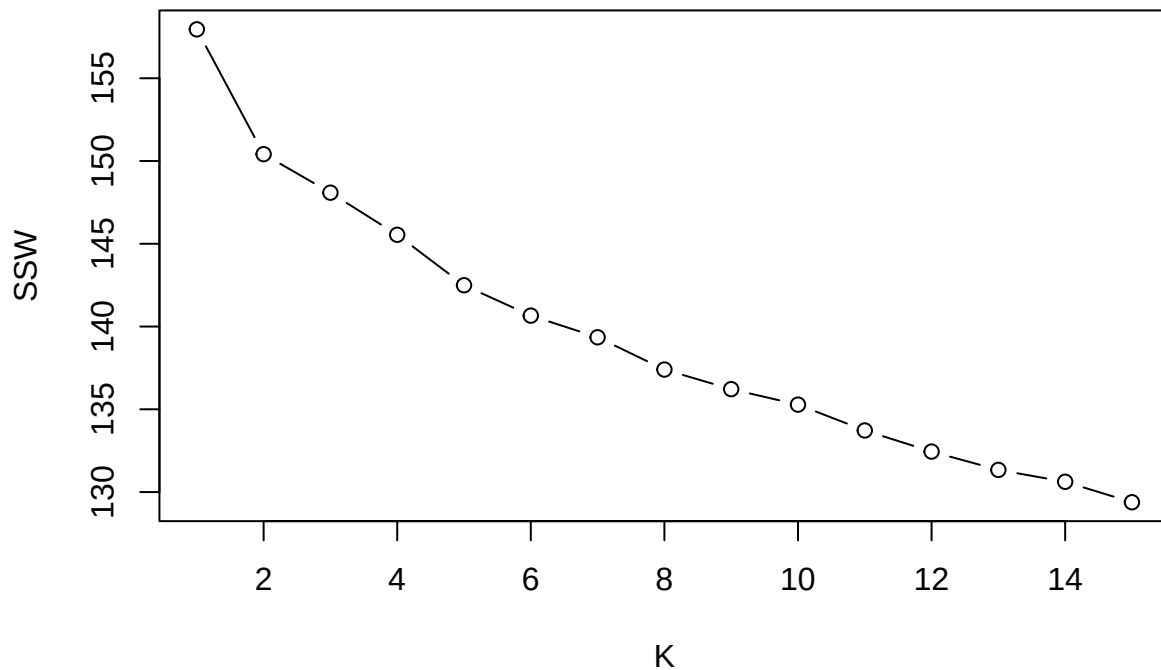
```
dim(mds.matrix) # Projected coordinates used by K-Means
```

```
## [1] 323 282
```

After projecting to Euclidean dimension, the Elbow Method is applied again in order to confirm that $K = 5$ is still the optimal amount of clusters for the dataset. The matrix that will be used will be the MDS Matrix in order to properly check for Euclidean geometry and cosine distance to see if that impacts anything with clustering.

```
# Redo elbow method:
n = 15
SSW = rep(0, n)
for (a in 1:n) {
  K = kmeans(mds.matrix, a, nstart = 20) # K-Means
  SSW[a] = K$tot.withinss # total within-cluster sum of squares
}
# Plot elbow:
plot(1:n, SSW, type = "b", xlab = "K", main = "K-Means Elbow Method (MDS)")
```

K-Means Elbow Method (MDS)



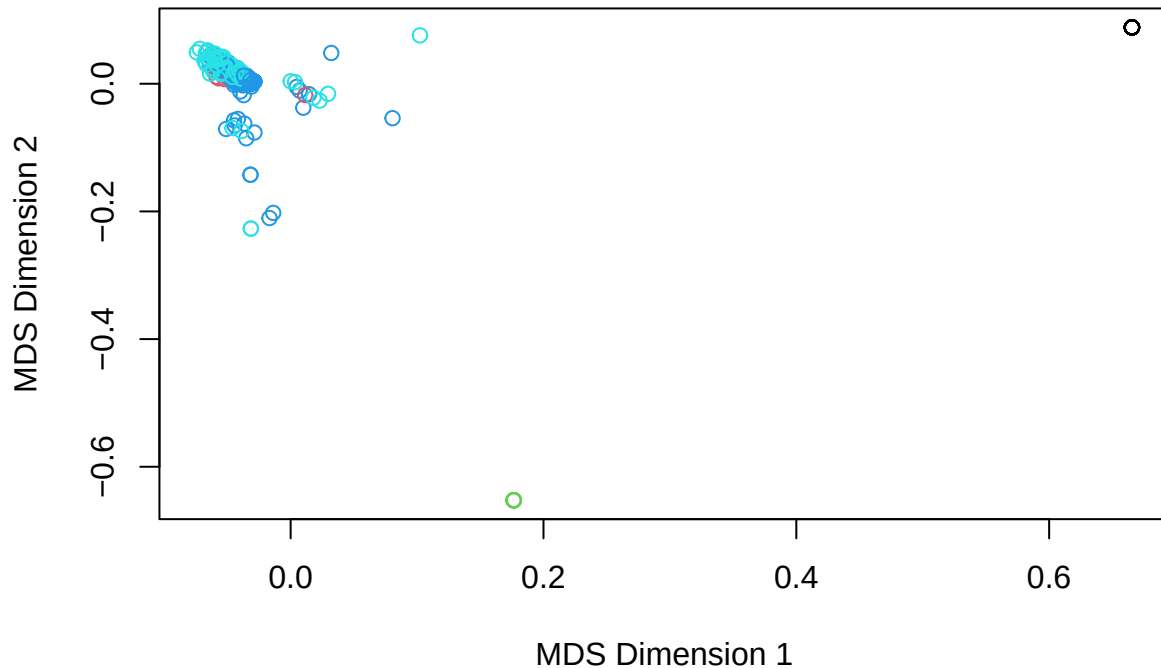
From the elbow method graph, the elbow begins to slow down at around $K = 5$, which confirms that 5 clusters is the ideal amount of clusters to be using for K-Means clustering, which is the value that will be used for the final clustering model for the posts dataset.

In order to plot the final graph, MDS will be applied to make two-dimensional visualisation possible. The colours shown in the plot are computed from higher-dimensional space instead of 2D space.

```
best_K = 5 # select K from elbow

# Plot clustering:
K = kmeans(mds.matrix, best_K, nstart = 20)
mds2.matrix = cmdscale(D, k = 2)
plot(mds2.matrix, col = K$cluster,
     xlab = "MDS Dimension 1", ylab = "MDS Dimension 2",
     main = "Visualisation of K-Means Clusters in Bluesky Posts")
```

Visualisation of K-Means Clusters in Bluesky Posts



From the visualisation, it illustrates that from the 5 clusters, there is little to no meaningful clusters that can be made based on similarities in their textual content as two of the clusters are basically outliers and the remaining clusters are very close together, suggesting that the clusters are very similar or very dissimilar to each other, with nothing between.

4.4) Examining K-Means Clusters Now that the clusters have been made, in order to actually prove if there is meaningful clusters, the clusters itself have to be examined and checked for its contents also.

Using the summary and table of K-Means clustering, the amount of values inside each cluster can be checked to see how the method divides each post into clusters to create meaningful clusters.

```
# Examine clusters:  
summary(K)
```

```
##           Length Class  Mode  
## cluster      323  -none- numeric  
## centers     1410  -none- numeric  
## totss         1  -none- numeric  
## withinss      5  -none- numeric  
## tot.withinss  1  -none- numeric  
## betweenss     1  -none- numeric  
## size          5  -none- numeric  
## iter          1  -none- numeric  
## ifault        1  -none- numeric
```

```
table(K$cluster)
```

```
##
##   1   2   3   4   5
## 17 19   7 222  58
```

As shown, most of the posts are grouped into one cluster which demonstrates little to no meaningful clusters based on similarities in textual content as the topics from the dataset are nearly the same all together.

Word Frequencies can be used to verify the content and themes of each cluster, so for this particular analysis, the top 10 words of each cluster in terms of frequency will be extracted for review to save on computation and readability and should be enough words to give a consistent theme of each cluster.

TF-IDF weighting matrix is used to calculate scores of each word for frequency and it is applied across every cluster when initially shaping the data.

```
# Finding top terms for each cluster:
for (i in 1:best_K) {
  clusterStr = paste("Cluster", i)

  clusterId = which(K$cluster == i)

  # Extract original TF-IDF matrix:
  clusterposts = text_matrix[clusterId, ]

  # Average each term's TF-IDF weight across documents:
  clusterTermWeight = colMeans(clusterposts)
  print(clusterStr)
  print(sort(clusterTermWeight, decreasing = TRUE)[1:10])
}
```

```
## [1] "Cluster 1"
##   bylin discount      fell      bill  percent      read  advertis  better
## 0.9248603 0.9248603 0.9248603 0.9111166 0.9111166 0.7512236 0.0000000 0.0000000
##   budget      expand
## 0.0000000 0.0000000
## [1] "Cluster 2"
##   code      audit      agent      secur      vulner      help      sanit      inject
## 0.4555336 0.2557326 0.1790140 0.1498844 0.1497036 0.1488310 0.1398644 0.1336829
##   astbas      can
## 0.1047038 0.1021140
## [1] "Cluster 3"
##   book      expir      three      two      watch      read  advertis  better
## 1.1382116 1.1382116 1.0524494 0.9893642 0.9549557 0.7512236 0.0000000 0.0000000
##   budget      expand
## 0.0000000 0.0000000
## [1] "Cluster 4"
## opensubstackcompubthebulwa      use
##      0.13759684      0.05848290
##      anthrop      applenewsaacblbqqui
##      0.04620566      0.04340822
##      incroy      japanaicojpmmedia
##      0.04340822      0.04340822
##      huh      olhardigitalcombri
```

```
##                0.04340822                0.04340822
##                say                artificialintellig
##                0.04161944                0.04050222
## [1] "Cluster 5"
##      googl      search      use      user      plan      bravia      hdr
## 0.34060348 0.06863608 0.06422190 0.06347435 0.05864982 0.05013008 0.05013008
##      inch      ole      soni
## 0.05013008 0.05013008 0.05013008
```

According to the top 10 words of each cluster, there exists separation between themes but there is not enough separation to confidently assign a theme to each cluster meaning that there is little to no evidence that meaningful themes can be made from just K-Means clustering.

Hierarchical Clustering

Hierarchical clustering is a method that groups data into a hierarchy of clusters that is based on how similar they are to each other and builds a dendrogram to visualise the relationship. This is another clustering method that will be applied to the Bluesky posts dataset through the sampling and normalisation of data to fit, calculation of cosine distance as the unit for clustering, and the variant of hierarchical clustering used will be single-linkage due to its ability to merge clusters based on similarity which is the goal of the analysis.

4.5) Data Preparation To begin Hierarchical Clustering, the data required for clustering to work would be need to be sampled which is done by selecting posts by group from the raw Bluesky posts data, extracting the raw text, then removing empty documents, and then selecting the first ten posts from each group. This allows a matrix to be created using the selected data which will be used for making the clusters in later steps.

```
# Load raw data:
load("../data/raw/bluesky_raw_data.RData")

# Extract texts only:
chatGPT_text = search_skeets_ChatGPT$text
claude_text = search_skeets_Claude$text
gemini_text = search_skeets_Gemini$text

# Group posts together (dendrogram post index for later examination)
posts = c(chatGPT_text, claude_text, gemini_text)

# Selecting balanced sample:
index = c(rep("CG", length(chatGPT_text)),
          rep("CL", length(claude_text)),
          rep("GE", length(gemini_text)))
index = index[-empties]

# Selecting the first 10 remaining posts of each group:
id.CG = which(index == "CG")[1:10]
id.CL = which(index == "CL")[1:10]
id.GE = which(index == "GE")[1:10]

# Make matrix and check the size
new.matrix = text_matrix[c(id.CG, id.CL, id.GE), ]
dim(new.matrix)
```

```
## [1] 30 2100
```

The matrix created will then be normalised similar to the step used when doing K-Means clustering which allows cosine distance to be calculated from the matrix. Row-names will be removed when normalising, so in order to prevent unnecessary data loss, the row-names of the non-normalised matrix will be pasted in into the new matrix.

```
# Normalising selected posts for cosine distances:
norm.new.matrix = diag(1 / sqrt(rowSums(new.matrix ^ 2))) %*% new.matrix

# Compare row names before restoration:
head(rownames(new.matrix))
```

```
## [1] "1" "5" "10" "18" "22" "24"
```

```
head(rownames(norm.new.matrix))
```

```
## NULL
```

```
# Restore row names:
rownames(norm.new.matrix) = rownames(new.matrix)
head(rownames(norm.new.matrix))
```

```
## [1] "1" "5" "10" "18" "22" "24"
```

4.6) Visualising Clusters After preparing the data, in order to execute hierarchical clustering by single-linkage, a distance matrix will have to be prepared using cosine distance off the normalised matrix in order to get the accurate distance of the Bluesky dataset. This will make the resulting plot be grouped by the most similar posts as the closest pair and the furthest pair will be the most dissimilar to each other which will visualise the clustering to be meaningful or not.

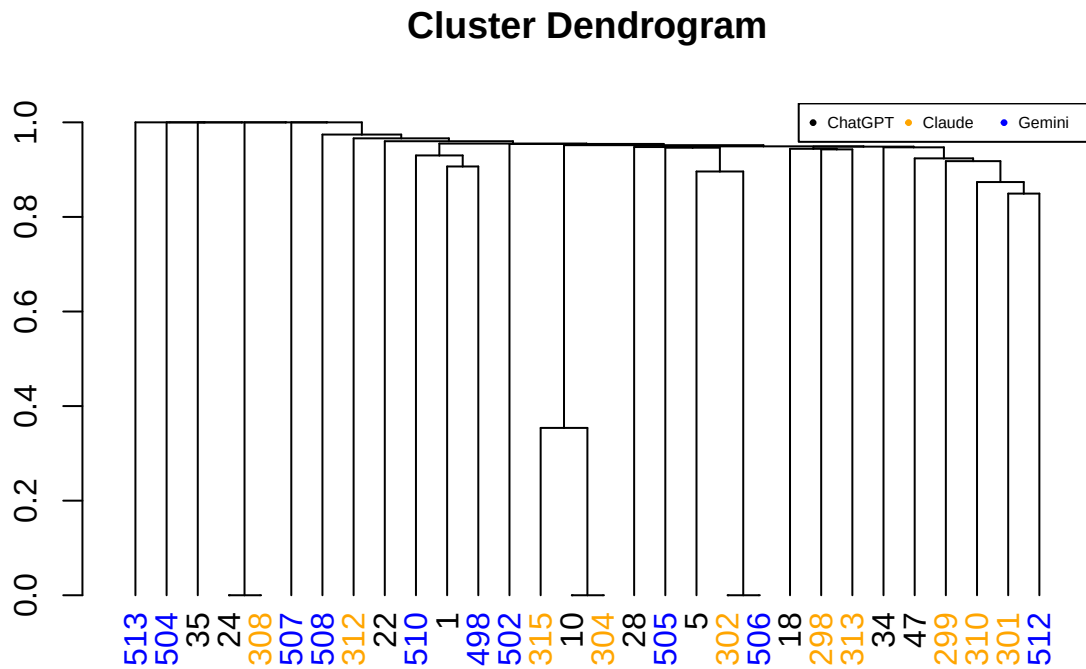
```
# Distance matrix from cosine distance:
D = dist(norm.new.matrix, method = "euclidean") ^ 2 / 2

# Single hierarchical clustering:
h = hclust(D, method = "single")
```

The visualisation will be a Dendrogram which is standard for hierarchical clustering and in order to make the plot clearer, custom colouring and legends are made for each group (ChatGPT, Claude, Gemini) as Black, Orange, and Blue in order to know what type of group the posts came from for reference.

```
# Apply colouring:
palette(c("black", "orange", "blue"))
dend = as.dendrogram(h)
colours = as.numeric(c(rep(1, 10), rep(2, 10), rep(3, 10)))
colours = colours[order.dendrogram(dend)]
labels_colors(dend) = colours

# Dendrogram:
plot(dend, main = "Cluster Dendrogram")
legend(x = "topright",
       legend = c("ChatGPT", "Claude", "Gemini"),
       col = c(1, 2, 3), pch = 20, ncol = 3, cex = 0.5)
```



From the clustering diagram, the posts do indicate some level of textual similarity but not very strong as the clusters from the dendrogram suggests a relatively weak level of similarity which do not form small numbers of clear, distinct groups which illustrates there is little to no evidence that meaningful groups can be formed by clustering from the Generative AI posts from both K-Means and Hierarchical clustering methods.

4.7) Examining Hierarchical Clusters While looking at the Dendrogram gives a suggestion of the level of similarity, we wouldn't know unless we examine what type of posts are grouped.

From the graph, Posts 24 and 308 were grouped together according to the clustering algorithm, so these two posts will be examined to check for similarity.

```
# Examining both post 24 and 308
posts[24]
```

```
## [1] " openAI chatGPT "
```

```
posts[308]
```

```
## [1] "Bureaucrats Rejoice, Claude for Government Has Arrived\n\nhttps://uenozooo.com/bureaucrats-rejoice"
```

While these two posts are textually similar, the subject/theme discussed about are not the same which suggests that textual similarity does not imply the same theme which implies that clustering may not have a strong correspondence to meaningful groups.

Posts 315 and 304 are grouped are a unique amount of distance that no other posts has, so these two will also be examined.

```
# Examining both post 315 and 304
posts[315]
```

```
## [1] "www.tiktok.com/t/ZTy91dbNd/"
```

```
posts[304]
```

```
## [1] "Links: abs, pdf\nSearch: Bluesky, Twitter, Reddit, Hacker News, Hugging Face, alphaXiv"
```

Upon inspection, both of these posts support the notion that textual similarity does not imply the same theme which still implies the weak evidence of meaningful clusters being generated from generative-AI posts based on similar text content.

Clustering: Conclusion

The results from both K-Means and Hierarchical Clustering prove that there is limited evidence of that Generative-AI posts can be grouped based on meaningful clusters based on textual similarity at all. The clusters were not clear enough to reveal strongly separated groups which suggest that the posts did not divide into categories based on features during analysis. Therefore, the findings found that there was weak or potentially no evidence at all that meaningful clusters can exist from just textual similarity in the dataset. However, this doesn't mean that there are no meaningful clusters at all, just the features selected were insufficient for clustering to make clear, distinct groups in the first place.

2.5 Network Analysis

The network analysis investigates which users are the most structurally important within a Bluesky follow network surrounding generative-AI discussions.

A node represents a Bluesky user and a directed edge represents one user following another. The graph is directed because User A following User B does not imply that User B follows User A.

Summary

The network analysis constructed a directed Bluesky follow network using a friends-of-friends approach. The final network contained 177 users and 182 directed follow relationships, with a low density of 0.0058, indicating a sparse structure. Centrality analysis showed that `bibivanderzee.bsky.social` had the highest degree, while `theguardian.com` had the highest closeness and betweenness centrality. These results identify the most connected and structurally influential users within the sampled network.

`atrrr` is used to access Bluesky data such as user profiles and follow relationships.

```
library(atrrr)
```

```
## Warning: package 'atrrr' was built under R version 4.5.3
```

`igraph` is used to build, visualise and analyse the network.

```
library(igraph)
```

```
## Warning: package 'igraph' was built under R version 4.5.3
```

```
##
```

```
## Attaching package: 'igraph'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      decompose, spectrum
```

```
## The following object is masked from 'package:base':
```

```
##
```

```
##      union
```

Load the raw Bluesky datasets collected earlier.

```
load("../data/raw/bluesky_raw_data.RData")
```

Combine the unique authors from all three AI topics. `unique()` removes users who appear more than once.

```
all_authors = unique(c(
  search_skeets_ChatGPT$author_handle,
  search_skeets_Claude$author_handle,
  search_skeets_Gemini$author_handle
))
```

Count the number of unique authors in the collected AI posts.

```
length(all_authors)
```

```
## [1] 549
```

Hence, there are 549 unique authors (our combined dataset).

Preview the first few authors.

```
head(all_authors)
```

```
## [1] "tmbhmltn.bsky.social"      "ai-prompts.bsky.social"
## [3] "nicheof.one"               "ayatawaka.bsky.social"
## [5] "kebrightbill.bsky.social"  "kazanomiya.bsky.social"
```

Retrieve profile information for every unique Bluesky author found in our ChatGPT, Claude and Gemini post data-sets.

`all_authors` contains 549 unique Bluesky handles. `get_user_info()` looks up each handle and returns profile information such as: `actor_handle`, `actor_name`, `followers_count`, `follows_count`.

```
author_info = get_user_info(all_authors)
```

Check how many author profiles were returned.

```
nrow(author_info)
```

```
## [1] 543
```

Sorting the authors by their number of followers. `order()` creates an ordering based on `followers_count`. `decreasing = TRUE` means the highest follower count comes first.

We then select only the columns that are useful for comparing users: - `actor_handle` = Bluesky username - `actor_name` = display name - `followers_count` = how many users follow them - `follows_count` = how many users they follow `head(..., 5)` keeps only the top 5 authors after sorting.

```
top_candidates = head(
  author_info[
    order(author_info$followers_count, decreasing = TRUE),
    c("actor_handle", "actor_name", "followers_count", "follows_count")
  ],
  5
)
```

Display the five most-followed authors. This lets us inspect the possible users that could be used as the starting point of the network.

```
top_candidates
```

```
## # A tibble: 5 x 4
##   actor_handle      actor_name      followers_count follows_count
##   <chr>           <chr>           <int>         <int>
## 1 theguardian.com The Guardian      793087         233
## 2 proptermalone.bsky.social ghost malone ergo pro~ 60164         4095
## 3 mikezaccardi.bsky.social Mike Zaccardi, CFA, C~ 35739          22
## 4 9to5mac.com      9to5Mac          31609          15
## 5 mack2114.bsky.social Mack Butterfield   31607         50189
```

Selecting the author with the highest follower count. `which.max()` finds the position of the largest value inside the `followers_count` column. We then use that position to retrieve the matching `actor_handle` (username). This author becomes the seed user, meaning the user we start from when building the follow network.

```
seed_user_handle = author_info$actor_handle[
  which.max(author_info$followers_count)
]
```

Display the selected seed user's Bluesky handle.

```
seed_user_handle
```

```
## [1] "theguardian.com"
```

These users form the first level of connections in our network.

```
friends_handles = get_follows(seed_user_handle, limit = 10)$actor_handle
```

```
## \ Got 10 follows, but there is more.. [239ms]
```

i Parsing

Display the accounts followed by the seed user (theGuardian.com).

```
friends_handles
```

```
## [1] "anabnos.com" "geraldinemckelvie.bsky.social"
## [3] "rorysymonsays.bsky.social" "jaseokundaye.bsky.social"
## [5] "laurasilver.bsky.social" "bibivanderzee.bsky.social"
## [7] "environment.theguardian.com" "hernandesraph.bsky.social"
## [9] "aliyaute.bsky.social" "rothandrew.bsky.social"
```

Check how many first-level users were returned.

```
length(friends_handles)
```

```
## [1] 10
```

Logical next steps: The Guardian -> 10 users The Guardian follows -> For each of those 10 users... -> retrieve up to 20 users they follow -> build the second level of the network (profile data) -> keep only the usernames -> use usernames to build network

Retrieve up to 20 accounts followed by each of the 10 first-level users. `lapply()` repeats `get_follows()` for every handle stored inside `friends_handles`. Each result contains the follow information for one first-level user.

```
more_friends = lapply(friends_handles, get_follows, limit = 20)
```

```
## \ Got 20 follows, but there is more.. [366ms]
## \ Got 20 follows, but there is more.. [353ms]
## \ Got 20 follows, but there is more.. [459ms]
## \ Got 20 follows, but there is more.. [433ms]
## \ Got 20 follows, but there is more.. [217ms]
## \ Got 20 follows, but there is more.. [332ms]
## i Parsing 3 results.v Got 3 results. All done!
## \ Got 20 follows, but there is more.. [238ms]
## i Parsing 9 results.v Got 9 results. All done!
## \ Got 20 follows, but there is more.. [453ms]
```

i Parsing
i Parsing
i Parsing
i Parsing
i Parsing
i Parsing
i Parsing
i Parsing
i Parsing

From each result, keep only the Bluesky usernames. `[[` extracts the `actor_handle` column from each `get_follows()` result. `more_friends_handles` will therefore contain one list of followed users for each first-level account.

```
more_friends_handles = lapply(more_friends, `[[`, "actor_handle")
```

Check how many second-level users were returned ...for each of the 10 first-level accounts.

```
sapply(more_friends_handles, length)
```

```
## [1] 20 20 20 20 20 20 3 20 9 20
```

Create edges from the seed user to the 10 first-level users.

Each row represents one directed follow relationship. “from” = the user doing the following. “to” = the user being followed.

The seed user is repeated once for every first-level user because ...The Guardian follows all 10 of these accounts.

```
el_seed = cbind(  
  from = rep(seed_user_handle, length(friends_handles)),  
  to = friends_handles  
)
```

Display these first-level relationships.

```
el_seed
```

```
##      from      to  
## [1,] "theguardian.com" "anabnos.com"  
## [2,] "theguardian.com" "geraldinemckelvie.bsky.social"  
## [3,] "theguardian.com" "rorysymonsays.bsky.social"  
## [4,] "theguardian.com" "jaseokundaye.bsky.social"  
## [5,] "theguardian.com" "laurasilver.bsky.social"  
## [6,] "theguardian.com" "bibivanderzee.bsky.social"  
## [7,] "theguardian.com" "environment.theguardian.com"  
## [8,] "theguardian.com" "hernandesraph.bsky.social"  
## [9,] "theguardian.com" "aliyaute.bsky.social"  
## [10,] "theguardian.com" "rothandrew.bsky.social"
```

Now creating the edges from each first-level user to their second-level users: Creating edges from each first-level user to the users they follow.

For each user: “from” = that first-level user’s handle. “to” = each account that user follows.

```
el_friends = lapply(  
  seq_along(friends_handles),  
  function(i) {cbind(  
    from = rep(friends_handles[i], length(more_friends_handles[[i]])),  
    to = more_friends_handles[[i]]  
  )  
}  
)
```

Combining the first-level and second-level relationships into one complete edge list.

do.call(rbind, el_friends) joins all the second-level edge tables together. rbind() then adds the seed-user edges. unique() removes any exact duplicate relationships.

```
el = unique( rbind( el_seed, do.call(rbind, el_friends)))
```

Remove self-follows We remove self-follows because we only want relationships between different users. Remove self-follows, where a user would point to itself.

```
el <- el[el[, "from"] != el[, "to"], , drop = FALSE]
```

Creating the full directed network from the completed edge list.

directed = TRUE is used because a follow has a direction: User A following User B does not mean User B follows User A.

```
g = graph_from_edgelist(el, directed = TRUE)
```

Display the graph object.

```
g
```

```
## IGRAPH 43d40a1 DN-- 177 182 --
## + attr: name (v/c)
## + edges from 43d40a1 (vertex names):
## [1] theguardian.com->anabnos.com
## [2] theguardian.com->geraldinemckelvie.bsky.social
## [3] theguardian.com->rorysymonsays.bsky.social
## [4] theguardian.com->jaseokundaye.bsky.social
## [5] theguardian.com->laurasilver.bsky.social
## [6] theguardian.com->bibivanderzee.bsky.social
## [7] theguardian.com->environment.theguardian.com
## [8] theguardian.com->hernandesraph.bsky.social
## + ... omitted several edges
```

Count the number of users/nodes in the network.

```
vcount(g)
```

```
## [1] 177
```

Count the number of follow relationships/edges in the network.

```
ecount(g)
```

```
## [1] 182
```

The directed graph contains 177 users (nodes) and 182 follow relationships (edges). The graph was successfully created from the edge list.

Full Network Visualisation

Each point represents a Bluesky user and each line/arrow represents a follow relationship.

`layout_with_fr()` positions connected users closer together so the network structure can be seen more clearly.

Node colours represent each user's role: - Yellow = seed user - Blue = first-level user - Green = second-level user

```
# Colour users based on their role in the network.
```

```
V(g)$color = ifelse(
  V(g)$name == seed_user_handle,
  "gold",
  ifelse(
    V(g)$name %in% friends_handles,
    "skyblue",
    "lightgreen"
  )
)
```

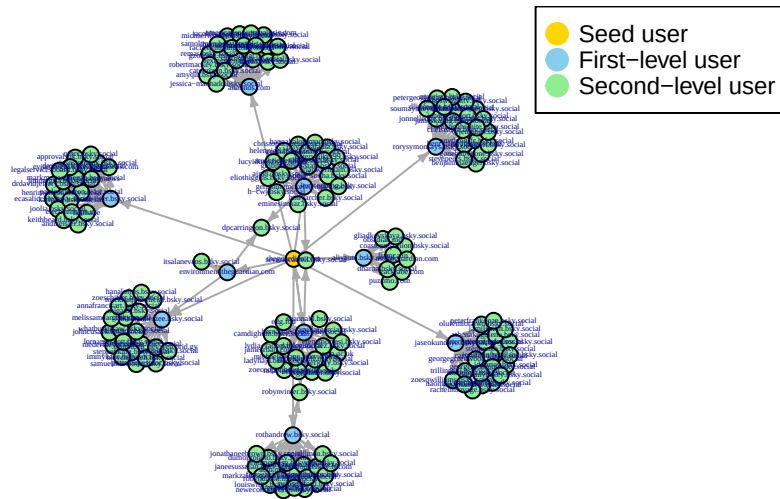
```
# Plot the full network.
```

```
plot(
  g,
  layout = layout_with_fr(g),
  vertex.size = 7,
  vertex.label.cex = 0.25,
  edge.arrow.size = 0.3,
  main = "Full Bluesky Follow Network"
)
```

```
# Add a key explaining the node colours.
```

```
legend(
  "topright",
  legend = c(
    "Seed user",
    "First-level user",
    "Second-level user"
  ),
  col = c(
    "gold",
    "skyblue",
    "lightgreen"
  ),
  pch = 19,
  pt.cex = 1.5,
  cex = 0.75,
  y.intersp = 0.9,
  bty = "o"
)
```

Full Bluesky Follow Network



Filtered Network Visualisation

The filtered network keeps users with more than one connection, making the more connected users easier to identify.

```
# Create a filtered subgraph containing users with more than one connection.
g2 = induced_subgraph(
    g,
    V(g)[degree(g) > 1]
)

# Plot the filtered network.
plot(
    g2,
    layout = layout_with_kk(g2),
    vertex.size = 15,
    main = "Key Connected Users in the Bluesky Filtered Follow Network"
)

# Add a key explaining the node colours.
legend(
    "topright",
    legend = c(
        "Seed user",
        "First-level user",
```

```

    "Second-level user"
  ),
  col = c(
    "gold",
    "skyblue",
    "lightgreen"
  ),
  pch = 19,
  pt.cex = 1.5,
  cex = 0.75,
  y.intersp = 0.9,
  bty = "o"
)

```

Key Connected Users in the Bluesky Filtered Follow Network



`dpcarrington.bsky.social`, `robynvinter.bsky.social`, and `severincarrell.bsky.social` are more connected than many second-level users that only appear once in the sampled network.

Network Properties

Network density: proportion of possible follow relationships that exist.

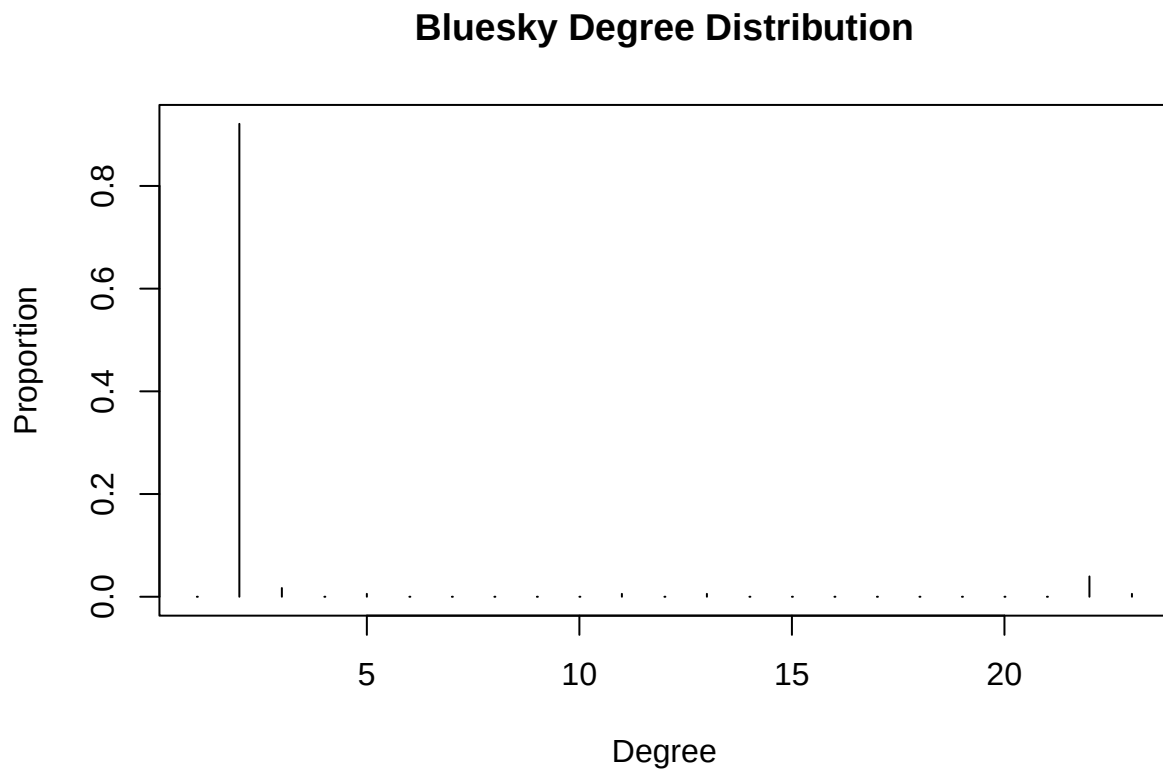
```
round(edge_density(g), 4)
```

```
## [1] 0.0058
```

Density = 0.0058, indicating a sparse network.

Degree distribution: shows how common different connection counts are.

```
plot(
  degree_distribution(g),
  type = "h",
  xlab = "Degree",
  ylab = "Proportion",
  main = "Bluesky Degree Distribution"
)
```



Most users have few connections, while a small number are more connected.

Centrality Measures

Degree: users with the most direct connections.

```
head(sort(degree(g), decreasing = TRUE), 5)
```

```
##      bibivanderzee.bsky.social      anabnos.com
##                               22                21
## geraldinemckelvie.bsky.social  rorysymonsays.bsky.social
##                               21                21
##      jaseokundaye.bsky.social
##                               21
```

bibivanderzee.bsky.social had the highest degree (22).

Closeness: users positioned closest to others through short paths.

```
head(sort(closeness(g, mode = "all"), decreasing = TRUE), 5)
```

```
##          theguardian.com      bibivanderzee.bsky.social
##          0.002923977          0.002109705
##          anabnos.com geraldinemckelvie.bsky.social
##          0.002096436          0.002096436
##      rorysymonsays.bsky.social
##          0.002096436
```

theguardian.com had the highest closeness score.

Betweenness: users acting as bridges between parts of the network.

```
head(sort(betweenness(g, directed = TRUE), decreasing = TRUE), 5)
```

```
##          theguardian.com      bibivanderzee.bsky.social      anabnos.com
##          323          80          60
##      rorysymonsays.bsky.social      jaseokundaye.bsky.social
##          60          60
```

theguardian.com had the highest betweenness score (323).

Network Analysis: Conclusion

Overall, the network analysis showed that a small number of users occupied more important structural positions, while most users had relatively few connections. However, the results are influenced by the sampled network design and selected seed user.

Overall Findings

Limitations

Network Analysis Limitations

This network is a sample rather than the complete Bluesky network. It was constructed from users appearing in the collected generative-AI posts.

The most-followed author was selected as the seed user, which influences the shape of the network and may also affect centrality results.

The network was limited to 10 first-level follows and up to 20 second-level follows for each first-level user.

Some user profiles or follow information may not have been returned by the Bluesky API, so the network may not include every possible connection.

The filtered graph g2 only keeps users with degree > 1 for clearer visualisation. Centrality measures were calculated on the full graph g.

Conclusion